SESSION 9

Software Architecture Design and Design Patterns

System Architecture Design – ExamEnv

Course	CS311P – Software Engineering and Design Principles Lab
Session	9
Project	ExamEnv
Deliverable	System Architecture Diagram

Prepared as the Session 9 practical submission.

1. Objective

To study software architecture and understand how a software system can be organized using layered architecture, client-server architecture, and the microservices concept. The architecture below is presented for the ExamEnv project.

2. System Architecture Diagram

The diagram gives a high-level view of the client, API, data services, and background/execution components.

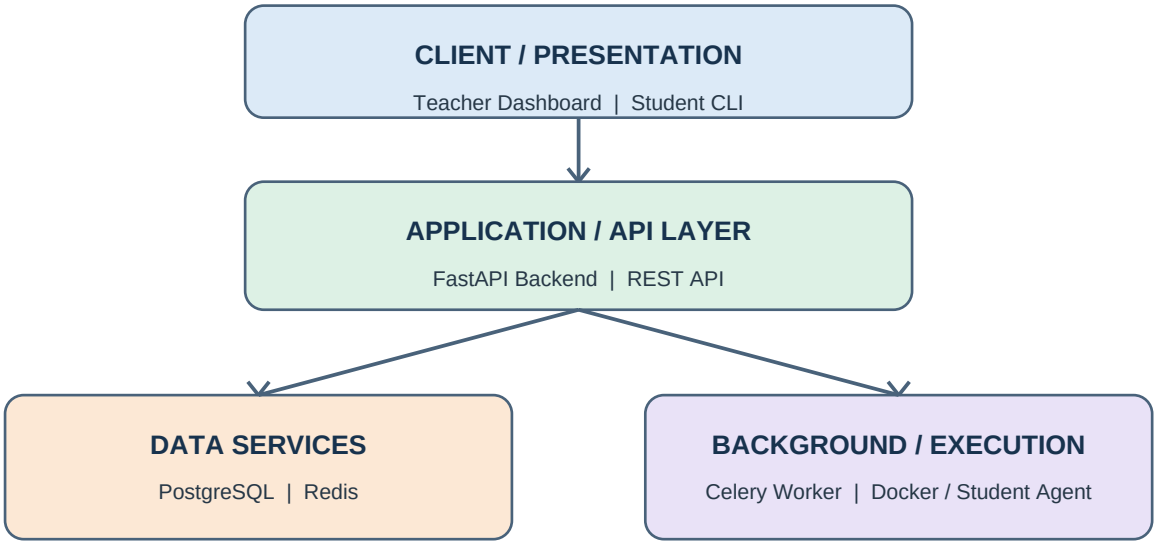

Figure 1. High-level architecture view of ExamEnv (conceptual grouping).

3. Layered Architecture Design

Presentation / Client Layer: Teacher Dashboard and Student CLI provide the user-facing interface.

Application / API Layer: FastAPI backend receives requests, applies application logic, and exposes REST endpoints.

Data Layer: PostgreSQL stores persistent application data, while Redis can support caching or queue-related data where configured.

Background / Execution Layer: Celery worker handles background tasks; Docker and the student agent represent the execution environment/components.

4. Client-Server Architecture

In the client-server model, clients send requests to a central server and receive responses. For ExamEnv, the Teacher Dashboard and Student CLI act as clients. The FastAPI backend acts as the server and communicates with the data and supporting services.

Typical request flow:

1. A user performs an action in the dashboard or CLI.
2. The client sends an HTTP request to the FastAPI REST API.
3. The backend processes the request and accesses required data/services.
4. The backend returns a response to the client.

5. Microservices Architecture (Conceptual)

Microservices architecture divides a system into small, independently manageable services. For this lab session, the following are conceptual service boundaries for discussing how ExamEnv could be organized. They do not imply that the repository currently deploys each item as a separate microservice.

Conceptual service	Responsibility
Authentication Service	User sign-in, identity, and access control.
Exam / Requirement Service	Receives and manages exam-related requirements.
Specification Service	Processes and maintains exam specifications.
Provisioning / Execution Service	Coordinates environment provisioning and execution.
Monitoring Service	Collects status and execution/health information.

6. Architecture Summary

ExamEnv can be explained using a layered view and a client-server interaction model. Microservices are included as a conceptual design discussion, as required by Session 9. The architecture diagram is a high-level representation and should be checked against the actual modules and configuration in the project repository before final submission.

7. Conclusion

This session demonstrates how software architecture helps organize system components, define their responsibilities, and explain how clients, backend services, data stores, and execution components interact. Layered and client-server views describe the system structure, while the microservices concept shows one possible way to separate responsibilities.

Deliverable: System Architecture Diagram

Note: This report is a high-level academic architecture description. Confirm exact service names and technologies with the team's implementation before submission.